QuickBites

A Food Delivery Platform

Assignment 3 — Transaction Behaviour, Crash Recovery & Concurrent Workload Testing

CS 432: Databases

Assignment 3, Module A & Module B

Semester II, 2025–2026

Team Members	Role
Bhavya Parmar	Module B Testing, Video and Report
Surriya Gokul	Module A Experiments
Sonagiri Thoshith Kautilya	Module A Report
Tanishq Chaudhari	Module B Concurrent Usage, Race Conditions Testing
Mohit	Module B Failure Simulation, Stress Testing

GitHub: github.com/mohit-codes-21/QuickBites
Video: Google Drive Link

Indian Institute of Technology, Gandhinagar

Contents

I Module A — Transaction Behaviour & Crash Recovery 3

1 Executive Summary 3

2 Project Objective and Scope 3
2.1 Objective (Module A) . . . 3
2.2 Key Assignment Constraints . . . 3

3 System Overview 3
3.1 Code Layout . . . 3
3.2 Data Model (Three Relations) . . . 4

4 Storage Engine: B+Tree as Primary Storage 4
4.1 Representation . . . 4
4.2 Operations . . . 4

5 Transaction Behaviour (BEGIN/COMMIT/ROLLBACK) 4
5.1 Where Transaction Logic Lives (Clarification Compliance) . . . 4
5.2 Undo Logging in the B+Tree . . . 5
5.3 Atomicity Across Multiple Relations . . . 5
5.4 Isolation (Serialized Execution) . . . 5

6 Consistency Guarantees 5
6.1 Schema and Check Constraints . . . 5
6.2 Foreign Key Constraints . . . 5

7 Durability and Crash Recovery 5
7.1 Write-Ahead Log (WAL) . . . 5
7.2 Persistence Strategy . . . 6
7.3 Recovery Algorithm . . . 6

8 Failure Simulation and ACID Validation 6
8.1 Multi-Relation Transaction Scenario . . . 6
8.2 Atomicity — Crash During Transaction . . . 6
8.3 Durability — Committed Data Survives Restart . . . 7
8.4 Consistency Test . . . 7
8.5 Isolation Test . . . 7

9 How to Reproduce (Module A) 8
9.1 Run Unit Tests . . . 8
9.2 Crash Scenario Runner . . . 8

10 Module A — Observations and Limitations 8

II Module B — Concurrent Workload & Stress Testing 9

11 Introduction 9

12 Verification Scope and Evaluation Criteria **9**

13 Implementation Controls for Correctness **9**
13.1 Transaction and rollback discipline 9
13.2 Concurrency and race hardening 9
13.3 Data integrity and observability 9

14 Experimental Setup and Test Scenarios **10**

15 Required Reliability Explanations **10**
15.1 How correctness of operations is ensured 10
15.2 How failures are handled 11
15.3 How multi-user conflicts are handled 11
15.4 What experiments were performed 11
15.5 Observations and limitations 11

16 Results and Analysis **12**
16.1 Scenario scale and created-entity counts 12
16.2 Suite-level outcomes 12
16.3 Detailed latest test entries 12
16.4 Stress and load comparison 13
16.5 Endpoint-level latency summary 14

17 Visual Evidence **14**

18 Discussion **17**

19 Reproducibility (Module B) **17**

Overall Conclusion **18**

Part I

Module A — Transaction Behaviour & Crash Recovery

1 Executive Summary

This part documents the Module A implementation, focusing on correctness and reliability under failures and concurrent execution. The system is a mini-database where each relation is stored in its own B+Tree; the B+Tree is the storage engine and the only access path. We implement multi-relation transactions with **BEGIN/COMMIT/ROLLBACK**, enforce constraints for **consistency**, and provide **write-ahead logging (WAL)** with **crash recovery** (redo committed effects and undo incomplete effects). Isolation is ensured via serialized transaction execution.

2 Project Objective and Scope

2.1 Objective (Module A)

The assignment objective is to make the system reliable and correct under both normal and heavy usage by:

- ensuring correct transaction behavior (all-or-nothing execution),
- implementing logging and crash recovery, and
- maintaining consistency of records stored in the B+Tree.

2.2 Key Assignment Constraints

The assignment explicitly requires that:

- **each relation is stored in a separate B+Tree**,
- **the primary key is the B+Tree key**,
- **the B+Tree value is the full record**, and
- **no separate copy of the data is maintained outside the B+Tree**.

3 System Overview

3.1 Code Layout

The Module A implementation lives in:

- `A3/Module A/db_management_system/database/bplustree.py` — B+Tree storage engine

- `A3/Module A/db_management_system/database/table.py` — relation abstraction (1 B+Tree per table)
- `A3/Module A/db_management_system/database/db_manager.py` — transaction coordinator + WAL + recovery
- `A3/Module A/db_management_system/tests/test_acid.py` — ACID and crash recovery verification

3.2 Data Model (Three Relations)

To satisfy the multi-relation transaction requirement, we use three tables:

Table	Attributes
Users	`user_id (PK), name, balance, city`
Products	`product_id (PK), name, stock, price`
Orders	`order_id (PK), user_id (FK), product_id (FK), amount`

4 Storage Engine: B+Tree as Primary Storage

4.1 Representation

For each relation, a dedicated B+Tree stores records as:

$$\text{key} = \text{primary key}, \qquad \text{value} = \text{full record (Python dict)}.$$

No heap file or auxiliary data store is used; the B+Tree is the database representation.

4.2 Operations

The engine supports point operations (search/insert/delete) and range scans through linked leaf nodes. The provided B+Tree implementation performs splits/merges to maintain B+Tree invariants.

5 Transaction Behaviour (BEGIN/COMMIT/ROLLBACK)

5.1 Where Transaction Logic Lives (Clarification Compliance)

Per the clarification received, rollback and transaction behavior are implemented **inside the B+Tree itself**. Concretely:

- Each B+Tree maintains a per-transaction **undo log** in-memory (before-images).
- Transactional writes are performed via `put_tx(txid, key, value)` and `delete_tx(txid, key)`.
- Rollback executes `rollback(txid)` which replays the undo log in reverse order.

The database manager coordinates a multi-table transaction by opening a tree-level transaction on each touched table.

5.2 Undo Logging in the B+Tree

For each B+Tree, the undo log stores tuples:

$$(\texttt{op}, \texttt{key}, \texttt{before_value})$$

where `before_value` is the previous record (or `None` if the key was absent).

5.3 Atomicity Across Multiple Relations

Atomicity for a multi-relation transaction is achieved by:

1. executing all updates/inserts/deletes through the tree-level transactional methods, and
2. on failure, rolling back each touched B+Tree using its native undo log.

5.4 Isolation (Serialized Execution)

Isolation is enforced by serializing transaction execution using a global re-entrant lock in the database manager. This yields a **serial schedule** and prevents concurrent corruption. While coarse-grained, it is sufficient for the assignment's "basic locking or serialized execution" requirement.

6 Consistency Guarantees

6.1 Schema and Check Constraints

Each table defines a schema with runtime validation (types, non-null requirements, and optional min/max bounds). Example constraints used in tests include:

- `Users.balance` ≥ 0
- `Products.stock` ≥ 0
- `Products.price` ≥ 0

Invalid writes raise an exception, triggering rollback.

6.2 Foreign Key Constraints

We support foreign key constraints in the database manager. At commit time, we validate that:

- `Orders.user_id` references an existing `Users.user_id`, and
- `Orders.product_id` references an existing `Products.product_id`.

If violated, the transaction is aborted and rolled back.

7 Durability and Crash Recovery

7.1 Write-Ahead Log (WAL)

Durability is implemented with a JSONL WAL file. For each transaction we log:

- `BEGIN(txid)`
- logical updates as `PUT/DELETE` including **before-images** and **after-images**
- `COMMIT(txid)` or `ABORT(txid)`

7.2 Persistence Strategy

The system persists table snapshots as JSON after enforcing the WAL rule:

1. ensure the WAL is synced (fsync) before writing table snapshots,
2. write table snapshots atomically (write-temp + rename), and
3. record `COMMIT` only after data is forced.

7.3 Recovery Algorithm

On restart, the system performs:

1. **REDO**: replay all `PUT/DELETE` operations in log order (idempotent upserts),
2. **UNDO**: for transactions without a `COMMIT`, undo using before-images in reverse log order,
3. validate constraints and foreign keys.

This satisfies the assignment requirement to undo incomplete transactions and preserve committed ones.

8 Failure Simulation and ACID Validation

8.1 Multi-Relation Transaction Scenario

We demonstrate a single transaction that touches all three relations simultaneously:

- deduct $10 from the user's balance (`Users` table),
- decrement the product stock by 1 (`Products` table),
- insert a new order record (`Orders` table).

8.2 Atomicity — Crash During Transaction

We simulate a crash after forcing dirty data to disk while the transaction is still uncommitted. After restart and recovery, the system correctly restores all three tables to their pre-transaction states. Table 1 shows the exact data observed at each stage.

Interpretation: The transaction modified all three tables and flushed the dirty pages to disk, but no `COMMIT` record was written to the WAL. On restart, the recovery algorithm redoes all logged operations then undoes the uncommitted transaction using before-images. The result is that *no partial updates remain*—atomicity is preserved.

Table 1: Atomicity validation: data states before, during, and after crash recovery.

HeaderGray **Variable**	**Before TX**	**During TX (on disk)**	**After Recovery**
`Users[1].balance`	100	90 (dirty)	**100** (restored)
`Products[101].stock`	5	4 (dirty)	**5** (restored)
`Orders[1000]`	—	exists (dirty)	— (removed)

8.3 Durability — Committed Data Survives Restart

We commit the same multi-relation transaction (deduct balance, decrement stock, insert order) and then restart the database from persistent storage. Table 2 shows exactly which values were committed, which were not, and whether each survived the restart.

Table 2: Durability validation: committed vs. uncommitted data across restart.

HeaderGray **Variable**	**Pre-Commit**	**Post-Commit**	**Committed?**	**Survived Restart?**
`Users[1].balance`	100	90	Yes	**Yes** (90)
`Products[101].stock`	5	4	Yes	**Yes** (4)
`Orders[1001]`	—	exists	Yes	**Yes** (exists)

Interpretation: All three committed changes persist after a full process restart. The WAL contains a `COMMIT` record for this transaction, so recovery replays the operations (redo) without undoing them. Durability is confirmed.

8.4 Consistency Test

We explicitly ran tests that attempted to violate consistency constraints:

Table 3: Consistency validation: constraint enforcement outcomes.

HeaderGray **Test Scenario**	**Constraint Violated**	**Outcome**
Insert order with `user_id=999`	FK: `Orders.user_id` → `Users.user_id`	Rejected + Rolled back
Set `Products[101].stock = -1`	Check: `stock` $\geq$ `0`	Rejected + Rolled back

After each rejected operation, the database state remained exactly as it was before the transaction began—no partial effects persisted.

8.5 Isolation Test

Two threads race to decrement the last unit of product stock (`stock=1`). Under serialized execution, exactly one transaction commits and the other aborts, leaving stock at 0 and exactly one order inserted. No intermediate state was visible to either thread.

9 How to Reproduce (Module A)

9.1 Run Unit Tests

From the repository root (with the project virtual environment), run:

```
python -m unittest discover -s "A3/Module A/db_management_system/tests"
-p "test_*.py"
```

9.2 Crash Scenario Runner

A crash is simulated via a helper that terminates the process mid-transaction:

```
python A3/Module A/db_management_system/tests/scenario_runner.py --data-dir
<DIR> --mode crash --flush
```

Restarting the database manager with the same data directory triggers recovery.

10 Module A — Observations and Limitations

Observation: During crash simulation and restart, logical REDO/UNDO recovery was reliable and deterministic; however, recovery time is expected to grow with the size of the WAL if it is allowed to grow indefinitely without checkpointing or log truncation.

- **Locking**: isolation is serial (global lock), not fine-grained.
- **Recovery granularity**: recovery is logical (redo/undo of operations), with periodic JSON snapshots.
- **Constraints**: schema checks, min/max bounds, and foreign keys are enforced; advanced constraints (e.g., unique secondary indexes) are out of scope.

Part II

Module B — Concurrent Workload & Stress Testing

11 Introduction

This part documents reliability verification for QuickBites under concurrent usage, race pressure, injected failures, and sustained load. The objective is to validate transaction behavior against ACID principles and assignment criteria: correctness of outcomes, rollback safety, multi-user isolation, and robustness under stress.

12 Verification Scope and Evaluation Criteria

The verification scope covers five suites: concurrent usage, race condition testing, failure simulation, ACID checks, and stress testing. Evaluation focused on four properties:

- **Atomicity:** operations either commit fully or rollback fully.
- **Consistency:** no invalid relational states, including orphan references or incomplete entities.
- **Isolation:** one user flow cannot corrupt or leak partial state into another.
- **Durability:** committed data persists across process restarts.

13 Implementation Controls for Correctness

13.1 Transaction and rollback discipline

Critical write paths execute in explicit transaction boundaries with commit-on-success and rollback-on-exception behavior. Failure tests intentionally inject partial-signup and partial-request conditions to verify that incomplete entities are not persisted.

13.2 Concurrency and race hardening

Race-sensitive operations are serialized using lock-backed techniques on shared resources such as customer carts and order assignment. Observed outcomes confirm one-winner behavior in race scenarios, with deterministic final state under contention.

13.3 Data integrity and observability

Database constraints and audit logs provide secondary integrity protection and post-run traceability. All suites emit structured metrics to CSV and JSONL for repeatable analysis and reporting.

14 Experimental Setup and Test Scenarios

Experiments were executed with Python test scripts and Locust against the local API service. The table below maps each suite to scenario intent and verification target.

HeaderGray **Suite**	**Primary Script**	**Scenario Focus**	**Verification Target**
Concurrent	`tests/concurrent_test.py`	Parallel reads and same-record writes	Isolation and non-interference
Race	`tests/race_condition_test.py`	Simultaneous signup and delivery acceptance	Single correct winner
Failure	`tests/failure_simulation_test.py`	Timeouts and forced disconnects	Rollback and no partial state
ACID	`tests/acid_verification.py`	Atomicity, consistency, isolation, durability	Property-level pass or fail
Stress	`tests/stress_test.py` and `locustfile.py`	High request volume and mixed CRUD	Correctness plus latency behavior

15 Required Reliability Explanations

15.1 How correctness of operations is ensured

Correctness is enforced through explicit transaction boundaries for critical writes, with commit on success and rollback on any failure path. Data integrity is further protected using relational constraints and post-operation validation checks. ACID verification scenarios confirm atomicity, consistency, isolation, and durability at the behavior level, not

only at code level. Structured logs and metrics provide traceability so each claim can be tied to reproducible evidence.

15.2 How failures are handled

Failure handling follows a fail-safe pattern: if a write cannot complete safely, the transaction is aborted and rolled back. Timeout and forced-disconnect simulations verify that partial entities are not left behind. Mid-transaction fault injection during signup confirms that invalid partial users cannot authenticate afterward. This approach ensures the system prefers rollback over inconsistent partial commit.

15.3 How multi-user conflicts are handled

Multi-user conflicts are managed using serialization controls on race-sensitive resources (for example, shared cart updates and delivery acceptance). In signup races, only one request is allowed to succeed for the contested identity while competing requests are rejected or rolled back. In delivery acceptance races, one-winner behavior is preserved and duplicate assignment is prevented. These checks were validated under concurrent thread pressure and repeated contention.

15.4 What experiments were performed

The following experiments were executed for this report:

- Concurrent usage: four scenarios including 15 parallel reads, 15 parallel updates, delete-while-readers, and 25-way cart contention.
- Race conditions: 50 concurrent signup contenders and 10 concurrent delivery acceptance contenders.
- Failure simulation: three injected fault modes (mid-transaction rollback, timeout handling, forced disconnect).
- ACID verification: atomicity, consistency, isolation, and durability checks including restart durability.
- Stress testing: threading workload of 1000 requests across 40 threads and Locust headless run with 60 users (latest aggregate 3652 requests).

15.5 Observations and limitations

Observed behavior is robust: all latest test entries passed, and both stress modes reported zero request failures in the latest run set. The main performance limitation is elevated tail latency on write-heavy paths compared with read-heavy paths under contention. Additional limitations include local-environment benchmarking (single-machine, development server context), synthetic workload distributions, and the absence of multi-node deployment effects. Therefore, the results are strong for assignment-grade reliability validation but should be complemented with production-style load profiling for final capacity planning.

16 Results and Analysis

16.1 Scenario scale and created-entity counts

To make concurrency and race outcomes explicit, Table 4 summarizes how many users, workers, contenders, or requests were involved in each scenario, and what entities were actually created versus rolled back.

These counts show that race-heavy user creation paths were stress-tested with high contention, while rollback behavior prevented duplicate or partial writes.

16.2 Suite-level outcomes

HeaderGray **Suite**	**Total**	**Passed**	**Failed**	**Pass Rate**
acid	5	5	0	100.00%
concurrent	4	4	0	100.00%
failure	4	4	0	100.00%
race	2	2	0	100.00%
stress	1	1	0	100.00%

Overall latest status: **16/16** test entries passed (**100.00%**).

16.3 Detailed latest test entries

HeaderGray **Test Case**	**Suite**	**Status**	**Duration (ms)**	**Parallel users/ops**	**Result Detail**
ACID Atomicity	acid	PASS	749.684	1 client write flow	Signup rejected as expected after injected fault; no partial user visible.
ACID Consistency	acid	PASS	532.453	1 client, 4 write ops	Integrity checks show zero orphan orders, order-items, and customers.
ACID Durability	acid	PASS	1425.692	1 commit + restart verify	Committed record remained visible after controlled service restart.
ACID Isolation	acid	PASS	423.972	2 parallel actors	No dirty visibility observed during partial write and disconnect window.
ACID Summary	acid	PASS	3421.197	Aggregate of 4 checks	Aggregate ACID status: 4/4 checks passed.
Concurrent Scenario 1 - 15 parallel reads	concurrent	PASS	866.930	15 parallel readers	All 15 reads completed successfully under parallel execution.
Concurrent Scenario 2 - 15 parallel updates	concurrent	PASS	924.887	15 parallel updaters	All updates succeeded; final state converged to a valid latest write.

HeaderGray **Test Case**	**Suite**	**Status**	**Duration (ms)**	**Parallel users/ops**	**Result Detail**
Concurrent Scenario 3 - delete while readers run	concurrent	PASS	887.844	14 readers + 1 deleter	Readers stayed stable while delete completed; post-delete visibility is zero.
Concurrent Scenario 4 - same cart item contention	concurrent	PASS	1045.663	25 parallel cart updates	Quantity updates serialized correctly; no corruption under 25 parallel writes.
Failure Simulation Summary	failure	PASS	2673.148	Aggregate of 3 tests	Failure suite aggregate status: 3/3 scenarios passed.
Failure Test 1 - Mid-transaction rollback	failure	PASS	790.622	1 signup operation	Signup aborted and rollback verified; account remains non-authenticatable.
Failure Test 2 - API timeout simulation	failure	PASS	937.499	1 timeout operation	Timeout handling preserved prior consistent state; no partial record written.
Failure Test 3 - Forced disconnect mid-write	failure	PASS	907.160	1 partial-write sender	Connection interruption did not leave partially committed user/account rows.
Race Condition - 50 concurrent signup attempts	race	PASS	12979.271	50 concurrent contenders	Exactly one successful signup; remaining contenders rolled back cleanly.
Race Condition - concurrent delivery acceptance	race	PASS	1515.869	10 delivery contenders	Single delivery partner accepted order; one-winner invariant maintained.
Stress Test (threading fallback)	stress	PASS	21939.766	40 threads, 1000 ops	1000 requests, 0.000% errors, avg 870.285 ms, median 670.651 ms, p95 2471.012 ms.

16.4 Stress and load comparison

HeaderGray **Metric**	**Threading Stress Script**	**Locust Headless Run**
Total requests	1000	3652
Error or failure rate	0.000%	0.000%
Average response time	870.285 ms	380.101 ms
P95 response time	2471.012 ms	840.000 ms
Median response time	670.651 ms	310.000 ms
Throughput	45.579 req/s	58.775 req/s

16.5 Endpoint-level latency summary

HeaderGray **Endpoint**	**Source**	**Avg (ms)**	**P95 (ms)**	**Error (%)**	**RPS**
GET /api/menu-items	Threading	881.842	1422.253	0.000	14.038
GET /api/menu-items?search=single	Threading	317.413	453.526	0.000	12.762
POST /api/menu-items	Threading	1624.641	3653.763	0.000	8.523
PUT /api/menu-items/<rid>/<iid>	Threading	991.745	2976.474	0.000	4.968
DELETE /api/menu-items/<rid>/<iid>	Threading	843.913	2596.822	0.000	5.287
DELETE /api/menu-items/<rid>/<iid>	Locust	297.508	510.000	0.000	10.976
GET /api/menu-items	Locust	525.616	870.000	0.000	14.806
GET /api/menu-items?search=single	Locust	222.189	370.000	0.000	16.046
POST /api/menu-items	Locust	518.796	1400.000	0.000	10.622
PUT /api/menu-items/<rid>/<iid>	Locust	274.287	450.000	0.000	5.359

17 Visual Evidence

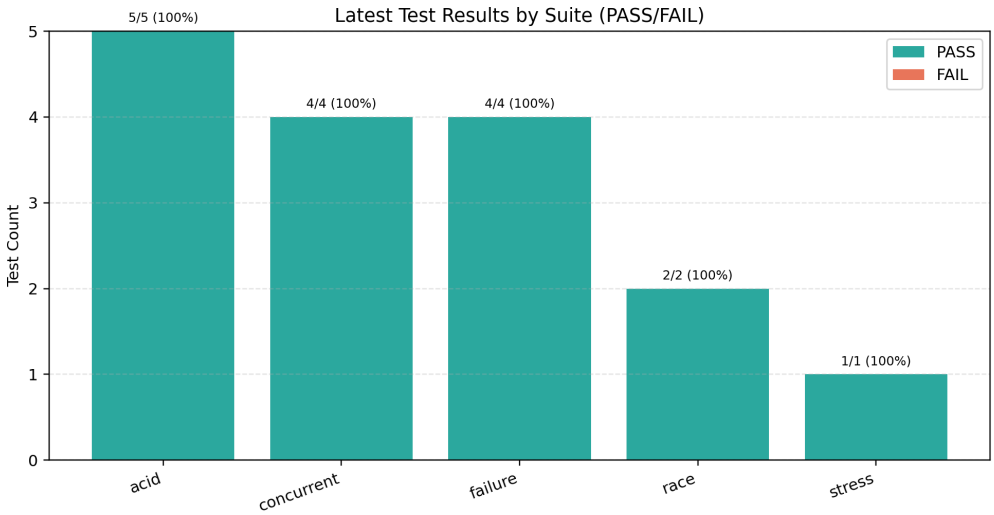

Figure 1: PASS and FAIL distribution by suite.

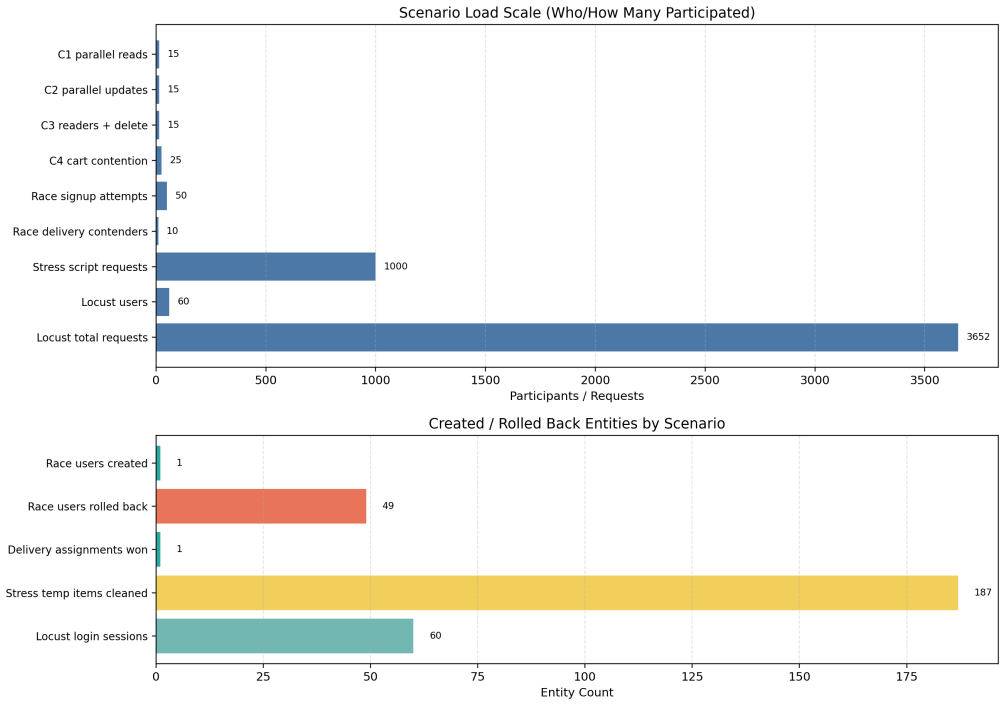

Figure 2: Scenario load scale and entity outcomes (participants, attempts, and created or rolled-back records).

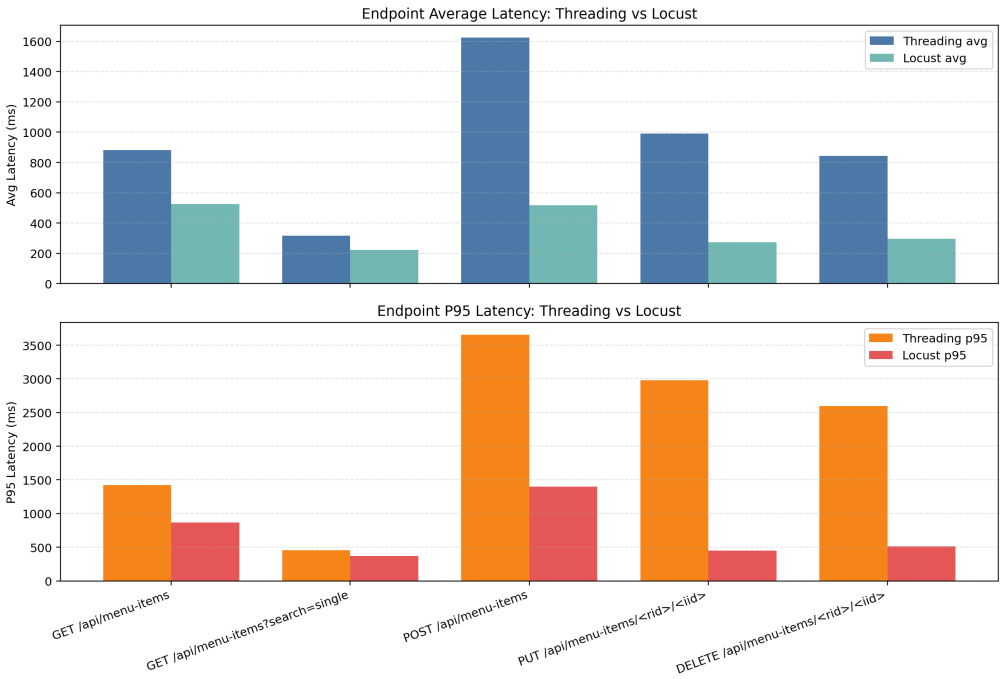

Figure 3: Endpoint latency comparison under stress (threading vs Locust, average and p95).

HeaderGray **Scenario**	**Load Scale**	**Created / Rolled Back**	**Observed Outcome**
Concurrent S1 (parallel reads)	15 reader threads	No new entities (read-only)	successful_reads = 15/15
Concurrent S2 (parallel updates)	15 updater threads	No new entities; one final item version	updates_ok = 15/15
Concurrent S3 (delete while reading)	14 readers + 1 deleter	No new entities; target record removed from active listing	readers = 14, post_visible = 0
Concurrent S4 (cart contention)	25 concurrent updates	No new users/items; cart quantity converged correctly	updates_ok = 25/25
Race signup	50 simultaneous signup attempts	Users created = 1, rolled back = 49	single active user row after race
Race delivery acceptance	10 delivery contenders	Winning assignments = 1	one-winner invariant maintained
Failure simulation suite	3 injected failure scenarios	Partial users/accounts created = 0	rollback preserved consistency in all 3 cases
Stress (threading script)	1000 requests across 40 threads	Temporary menu records created and cleaned = 187	error_rate = 0.000%
Stress (Locust headless)	60 simulated users, 3652 requests	Login sessions observed = 60, failures = 0	sustained mixed CRUD with 0 failures

Table 4: Scenario participation scale and created-entity outcomes from latest run artifacts.

18 Discussion

The latest execution indicates strong transactional correctness across concurrent, race, failure, and ACID suites. All suites reported complete pass status with no residual integrity anomalies. Under stress, correctness remained stable (0% errors in both threading and Locust runs), but write-heavy operations exhibit materially higher tail latency than read-oriented operations. This pattern is consistent with lock contention and transaction serialization on critical write paths.

Locust appears faster than the threading stress script primarily because workload shape differs: Locust applies user think-time and distributed task pacing, while the threading script continuously drives requests with fixed worker concurrency and more sustained contention bursts. Therefore, Locust and threading values should be interpreted as complementary profiles, not direct one-to-one latency replacements.

19 Reproducibility (Module B)

Latest verification can be reproduced with:

```
python tests/concurrent_test.py
python tests/race_condition_test.py
python tests/failure_simulation_test.py
python tests/acid_verification.py
python tests/stress_test.py
locust -f locustfile.py --headless -u 60 -r 20 -t 1m --host http://127.0.0.1:5001
```

Overall Conclusion

Module A demonstrates that the B+Tree-backed mini-database satisfies all ACID requirements: multi-relation transactions provide atomicity (verified via crash simulation and recovery); schema validation, check constraints, and foreign keys provide consistency; serialized execution provides isolation; and WAL with redo/undo recovery provides durability across crashes. All data states were verified with exact before/during/after comparisons.

Module B confirms that the full-stack QuickBites application maintains transactional reliability under concurrent workload and stress conditions. All 16 test entries across 5 suites passed with 100% success rate. Race conditions, failure injections, and high-volume stress tests produced zero data-integrity violations and zero request failures.

Together, these modules demonstrate end-to-end reliability from the storage engine level (Module A) through the application API level (Module B), meeting all requirements of Assignment 3.